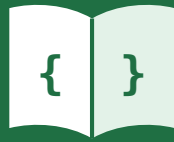

Modül 10 — Elasticsearch ile Arama

Ders Notları

Spring Boot Edu



2026-09-23

İçindekiler

1. Öğrenme Hedefleri	2
2. Kavramlar	2
2.1 Neden Bir Arama Motoru?	2
2.2 Analyzer'lar	2
2.3 text mi, keyword mü?	3
2.4 Elasticsearch Asıl Veri Kaynağı Değildir	3
3. Adım Adım Örnekler	3
3.1 Belgeyi Eşlemek	4
3.2 Repository	5
3.3 NativeQuery ile Tam Metin Arama	5
3.4 Aggregation'larla Facet'ler	6
3.5 Arama API'si	7
3.6 İndeksi PostgreSQL ile Senkron Tutmak	8
3.7 Testcontainers ile Test	9
4. Sık Yapılan Hatalar ve En İyi Pratikler	10
5. Özet	10
6. İleri Okuma	11

1. Öğrenme Hedefleri

Bu modülün sonunda şunları yapabileceksiniz:

- Ters indeksin (inverted index) ve analyzer'ların tam metin aramayı nasıl hızlandırdığını açıklamak
- Bir Java record'unu @Document ve @Field ile, Türkçe analyzer dahil, bir indekse eşle-mek
- CRUD ve türetilmiş sorgular için ElasticsearchRepository kullanmak
- NativeQuery ile bool sorgular yazmak: tam metin eşleşme, alan ağırlıkları, filtreler ve highlight
- Aggregation'larla kategori başına sonuç saymak (facet)
- Arama indeksini her commit'ten sonra PostgreSQL ile senkron tutmak
- Arama kodunu @DataElasticsearchTest ve Testcontainers ile test etmek

Ön koşullar: Modül 06 (PostgreSQL, transaction'lar) · **Tahmini süre:** 5 saat · **Docker gerekir** (Elasticsearch için Docker'a en az 2 GB bellek verin)

2. Kavramlar

2.1 Neden Bir Arama Motoru?

PostgreSQL'de WHERE description LIKE '%kitapları%' her satırı okur, “Kitapları” (bü-yük/küçük harf) ve “kitaplar” (aynı kelimenin başka bir biçimi) kayıtlarını kaçıır ve hangi sonucun en uygun olduğunu söyleyemez. Bir arama motoru bunu **ters indeks** ile çözer: Bir belge saklanırken metni terimlere bölünür ve indeks her terim için onu içeren belge-leri hatırlar.

```
"Yazılım mimarisi üzerine kitaplar" → yazıl · mimaris · üzer · kitap
                                     |
ters indeks: kitap → [belge 3]
               mimaris → [belge 3]
```

Arama, sorgu metnini aynı şekilde analiz eder ve terimleri indekste arar. Her sonuç bir **puan** (BM25) alır: Nadir terimler ve kısa alanlar daha çok sayılır.

2.2 Analyzer'lar

Bir analyzer metni terimlere dönüştürür: kelimelere böler (tokenizer), küçük harfe çevirir, etkisiz kelimeleri (stop words) atar, kelimeleri köklerine indirger.

Metin	standard analyzer	turkish analyzer
İstanbul'daki	istanbul'daki	istanbul
kitapçılar	kitapçılar	kitapçı
IŞIK	işık (yanlış harf)	ışık

turkish analyzer Türkçe küçük harf kurallarını bilir (I → ı, i değil), kesme işaretinden sonraki eki atar ve çekim eklerini keser. Kökler gerçek kelime olmak zorunda değildir: Metin ve sorgu için aynı olmaları yeterlidir. Analyzer'ı her alan için, içeriğinin diline göre seçin.

2.3 text mi, keyword mü?

text	keyword
Terimlere analiz edilir	Tek ve kesin bir değer olarak saklanır
Tam metin arama, alaka puanı	Filtreler, sıralama, aggregation'lar
Başlık, açıklama	Kategori, ISBN, yazar, durum

Bir alan ikisi birden olabilir: keyword alt alanı olan bir text alanı (multi-field).

2.4 Elasticsearch Asıl Veri Kaynağı Değildir

Elasticsearch'te belgeler arası transaction yoktur ve yeni bir belge ancak bir sonraki **refresh**'ten sonra aranabilir olur (varsayılan olarak saniyede bir). Veriyi PostgreSQL'de tutun ve indeksi her an yeniden kurulabilen bir kopya olarak görün.

3. Adım Adım Örnekler

Docker açıkken uygulamayı başlatın. PostgreSQL ve Elasticsearch, kök dizindeki `compose.yaml` dosyasından (elastic ve postgres profilleri) başlar:

```
./mvnw -pl modules/10-elasticsearch/lesson spring-boot:run
```

Tur (LessonTour), açılışta altı kitabı PostgreSQL'den Elasticsearch'e kopyalar ve aşağıdaki örneklerin hepsini çalıştırır.

i Not

Bu modülün tabloları PostgreSQL'in search şemasında durur (spring.flyway.schemas ve spring.datasource.hikari.schema). Böylece ortak bookstore veritabanındaki modül 05 ve 06 tablolarıyla asla çakışmaz.

3.1 Belgeyi Eşlemek

```
@Document(indexName = "books")
public record BookDocument(
    @Id String id,
    @MultiField(mainField = @Field(type = FieldType.Text), //
        ↪ searchable words ...
        otherFields = @InnerField(suffix = "sort", type = FieldType.Keyword))
    ↪ // ... and sortable as a whole
    String title,
    @Field(type = FieldType.Keyword) String author, //
    ↪ exact values only
    @Field(type = FieldType.Text, analyzer = "turkish") String description,
    ↪ // Turkish stemming + lowercase
    @Field(type = FieldType.Keyword) String category, //
    ↪ filters and facets
    @Field(type = FieldType.Double) double price) {

    public static BookDocument from(Book book) {
        return new BookDocument(String.valueOf(book.id()), book.title(), book.author(),
            ↪ book.description(),
            book.category(), book.price().doubleValue());
    }
}
```

- @Document(indexName = "books"): İndeks henüz yoksa Spring Data, repository başlarken indeksi bu eşlemeyle oluşturur.
- title bir multi-field'dır: tam metin arama için title, sıralama için keyword olarak title.sort.
- description alanı turkish analyzer kullanır, çünkü açıklamalar Türkçedir.
- author ve category keyword'dür: filtreler ve facet'ler için kesin değerler.

Spring Data'nın oluşturduğu eşlemeye bakın:

```
curl localhost:9200/books/_mapping
```

Spring Data ayrıca bir _class alanı ekler. Belgeleri doğru Java tipine geri okumak için bunu kullanır.

Analyzer'ı doğrudan deneyin:

```
curl -XPOST localhost:9200/_analyze -H 'Content-Type: application/json' \
  -d '{ "analyzer": "turkish", "text": "İstanbul\'daki kitapçılar" }'
# terimler: "istanbul", "kitapçı"
```

3.2 Repository

```
public interface BookSearchRepository extends ElasticsearchRepository<BookDocument,
    ↪ String> {

    List<BookDocument> findByAuthor(String author);           // term query on the
    ↪ keyword field
}
```

ElasticsearchRepository, modül 05-07'deki repository'ler gibi çalışır: save, findById, deleteAll, sayfalama ve türetilmiş sorgular. findByAuthor, keyword alanı author üzerinde bir term sorgusuna dönüşür.

3.3 NativeQuery ile Tam Metin Arama

Gerçek arama sayfaları için türetilmiş sorgular yetmez. NativeQuery, Elasticsearch Java client'ının tüm sorgu dilini sunar:

```
public List<BookHit> search(String text, @Nullable String category, @Nullable Double
    ↪ maxPrice) {
    NativeQuery query = NativeQuery.builder()
        .withQuery(q -> q.bool(bool -> {
            bool.must(m -> m.multiMatch(match -> match
                .query(text)
                .fields("title^3", "description")));           // a title
            ↪ match counts 3x
            if (category != null) {
                bool.filter(f -> f.term(t -> t.field("category").value(category)));
            }
            ↪ // yes/no, no score
            if (maxPrice != null) {
                bool.filter(f -> f.range(r -> r.number(n ->
            ↪ n.field("price").lte(maxPrice))));
            }
            return bool;
        })))
        .withHighlightQuery(new HighlightQuery(new Highlight(List.of(
            new HighlightField("title"), new HighlightField("description")),
            ↪ BookDocument.class)))
```

```

        .build();
    return operations.search(query,
        ↪ BookDocument.class).stream().map(BookHit::from).toList();
}

```

- Bir **bool** sorgu cümleleri birleştirir. must cümleleri eşleşmek zorundadır ve puana eklenir. filter cümleleri de eşleşmek zorundadır, ancak puanı değiştirmez ve Elasticsearch onları önbelleğe alabilir.
- `multi_match` birden çok alanda arar. `title^3`, başlıktaki bir eşleşmenin üç kat sayılmasını sağlar.
- Highlight isteği, eşleşen kelimelerin etrafına `<em>` etiketi konmuş parçaları döndürür.

Tur çıktısı (puan parantez içinde):

```

== 3.3 Full-text search
java: [Effective Java (3.36), Java Puzzlers (3.36), Spring in Action (0.65)]
kitap (text says "kitaplar"): [Clean Architecture (1.45)]
istanbul (text says "İstanbul"): [İstanbul Hatırası (5.03)]
java, programming, <= 60: [Java Puzzlers (3.36)]
highlight: [İstanbul sokaklarında geçen bir <em>polisye</em> roman.]

```

“Spring in Action” kitabında “Java” yalnızca açıklamada geçer, bu yüzden başlığında “Java” olan iki kitaptan çok daha düşük puan alır.

3.4 Aggregation’larla Facet’ler

Bir arama sayfası genellikle her kategoride kaç sonuç olduğunu gösterir. Bir **aggregation** bunu aynı istekte hesaplar:

```

public Map<String, Long> categoryFacets(String text) {
    NativeQuery query = NativeQuery.builder()
        .withQuery(q -> q.multiMatch(match -> match.query(text).fields("title^3",
            ↪ "description")))
        .withAggregation("categories", Aggregation.of(a -> a.terms(t ->
            ↪ t.field("category"))))
        .withMaxResults(0) // only the counts,
            ↪ no documents
        .build();
    SearchHits<BookDocument> hits = operations.search(query, BookDocument.class);

    var aggregations = (ElasticsearchAggregations)
        ↪ Objects.requireNonNull(hits.getAggregations());
    List<StringTermsBucket> buckets =
        ↪ Objects.requireNonNull(aggregations.get("categories"))
            .aggregation().getAggregate().sterms().buckets().array();
}

```

```

Map<String, Long> counts = new LinkedHashMap<>(); // biggest category
    ↪ first
buckets.forEach(bucket -> counts.put(bucket.key().stringValue(), bucket.docCount()));
return counts;
}

```

- terms aggregation'ı belgeleri bir keyword alanının değerlerine göre gruplar, en büyük grup önce gelir.
- withMaxResults(0) belgeleri değil, yalnızca sayıları döndürür.

```
== 3.4 Facets
```

```
java per category: {programming=3}
```

3.5 Arama API'si

```

@RestController
@RequestMapping("/api/search")
class SearchController {

    private final BookSearch search;

    SearchController(BookSearch search) {
        this.search = search;
    }

    @GetMapping
    SearchResponse search(@RequestParam String q,
                        @RequestParam(required = false) @Nullable String category,
                        @RequestParam(required = false) @Nullable Double maxPrice) {
        return new SearchResponse(search.search(q, category, maxPrice),
    ↪ search.categoryFacets(q));
    }
}

```

```
curl 'localhost:8080/api/search?q=java&maxPrice=90'
```

```

{"hits":[{"id":"1","title":"Effective Java", ... ,"highlights":["<em>Java</em> dilinde
    ↪ etkili programlama için doksan kural.", "Effective <em>Java</em>"]},
    {"id":"5","title":"Java Puzzlers", ...}],
"categories":{"programming":3}}

```

Sonuçlar maxPrice'a uyar (2 kitap), ancak facet "java" için 3 eşleşmenin hepsini sayar, çünkü categoryFacets filtreleri bilmez. Ödev 2'de bunun bilinçli ve doğru yapıldığı bir facet araması yazacaksınız.

3.6 İndeksi PostgreSQL ile Senkron Tutmak

Katalog PostgreSQL'e yazılır. Her yazma, transaction içinde bir BookSaved event'i yayınlar:

```
@Service
public class BookStore {

    private final JdbcClient jdbc;
    private final ApplicationEventPublisher events;

    public BookStore(JdbcClient jdbc, ApplicationEventPublisher events) {
        this.jdbc = jdbc;
        this.events = events;
    }

    @Transactional
    public Book add(NewBook book) {
        Book saved = insert(book);
        events.publishEvent(new BookSaved(saved));           // delivered after the
        ↪ commit (BookIndexer)
        return saved;
    }
}
```

Bir @TransactionalEventListener event'i **yalnızca commit'ten sonra** alır:

```
@Component
public class BookIndexer {

    private final BookSearchRepository index;
    private final BookStore store;

    public BookIndexer(BookSearchRepository index, BookStore store) {
        this.index = index;
        this.store = store;
    }

    @TransactionalEventListener                               // default phase:
    ↪ AFTER_COMMIT
    public void onBookSaved(BookSaved event) {
        index.save(BookDocument.from(event.book()));          // rolled-back
        ↪ books never get here
    }

    public long reindexAll() {                                // rebuild the
        ↪ index from the source of truth
        index.deleteAll();
    }
}
```

```

        index.saveAll(store.findAll().stream().map(BookDocument::from).toList());
        return index.count();
    }
}

```

- Geri alınan (rollback) bir transaction indekse asla ulaşmaz. BookIndexSyncIT bunu kontrol eder: addAll yeni bir kitap ekler, sonra tekrarlanan bir ISBN'de başarısız olur ve yeni kitap hiçbir zaman aranabilir olmaz.
- reindexAll, indeksi PostgreSQL'den yeniden kurar. Tur bunu açılıştta çalıştırır.

! Uyarı

Commit'ten sonra artık hiçbir şey geri alınamaz. Elasticsearch o anda kapalıysa event kaybolur ve indekste kitap eksik kalır. Düzenli yeniden indeksleme bunu onarır. Garanti için event'i aynı transaction içinde bir **outbox** tablosuna yazın ve bir mesaj aracısıyla iletin (modül 11, Kafka).

3.7 Testcontainers ile Test

```

@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    static final ElasticsearchContainer ELASTICSEARCH = new ElasticsearchContainer(
        DockerImageName.parse("elasticsearch:9.4.5")

        ↪ .asCompatibleSubstituteFor("docker.elastic.co/elasticsearch/elasticsearch"))
        .withEnv("xpack.security.enabled", "false") // plain HTTP, no
        ↪ password: tests only
        .withEnv("ES_JAVA_OPTS", "-Xms512m -Xmx512m");

    static final PostgreSQLContainer POSTGRES = new PostgreSQLContainer(
        DockerImageName.parse("pgvector/pgvector:0.8.6-
    ↪ pg18").asCompatibleSubstituteFor("postgres"));

    @Bean
    @ServiceConnection
    ElasticsearchContainer elasticsearchContainer() {
        return ELASTICSEARCH;
    }

    @Bean
    @ServiceConnection
    PostgreSQLContainer postgresContainer() {
        return POSTGRES;
    }
}

```

}

- Compose'daki image'ın adı `elasticsearch`, `Testcontainers` sınıfı ise `docker.elastic.co/elasticsearch` bekler: `asCompatibleSubstituteFor` ikisini bağlar.
- `xpack.security.enabled=false`, `compose.yml`'daki gibi parolasız düz HTTP sağlar. Yerel geliştirme dışında bunu asla yapmayın.
- `@DataElasticsearchTest` yalnızca Elasticsearch altyapısını ve repository'leri yükler. `BookSearchTest`, belgelerini `@BeforeEach` içinde kaydeder ve `indexOps(...).refresh()` çağırır. Böylece belgeler hemen aranabilir olur.

★ İpucu

Refresh olmadan bir test, Elasticsearch yeni belgeleri görünür yapmadan önce arama yapabilir ve yalnızca bazen başarısız olur.

4. Sık Yapılan Hatalar ve En İyi Pratikler

!! Dikkat

Elasticsearch'ü tek veritabanınız olarak kullanmayın. Asıl veriyi transaction destekli bir veritabanında tutun ve indeksi ondan yeniden kurulabilir yapın.

- **Yapın:** Filtrelediğiniz, sıraladığınız veya aggregation yaptığınız değerler için keyword seçin. Bir text alanında aggregation başarısız olur.
- **Yapmayın:** Sıralamayı değiştirmemesi gereken koşulları `must` içine koymayın. `filter` kullanın: daha hızlıdır ve ön belleğe alınabilir.
- **Yapın:** Analyzer'ı alanın diline göre seçin. `standard analyzer` ile Türkçe metin çekimli biçimleri kaçırmaz ve `I` harfini `ı` yerine `i` yapar.
- **Yapmayın:** Var olan bir alanın eşlemesini değiştirmeyin. Yeni eşlemeyle yeni bir indeks oluşturup yeniden indeksleyin (Ödev 3).
- **Yapın:** Testlerde yazdıktan sonra indeks refresh edin. Üretimde ise asla her yazmadan sonra refresh çağırmayın.
- **Yapmayın:** Güvenliği (`xpack.security.enabled=false`) kendi bilgisayarınız dışında hiçbir yerde kapatmayın.

5. Özet

- Ters indeks ve analyzer'lar tam metin aramayı hızlı ve dile duyarlı yapar. Her sonucun bir alaka puanı vardır.
- `@Document` ve `@Field` eşlemeyi tanımlar: arama için text, filtre ve facet için keyword, alan başına analyzer.

- ElasticsearchRepository CRUD'u ve basit sorguları karşılar. NativeQuery bool sorgular, ağırlıklar, filtreler ve highlight sunar.
- Aggregation'lar facet'leri aynı istekte hesaplar.
- Asıl veri kaynağı PostgreSQL olarak kalır. @TransactionalEventListener commit'ten sonra indeksler, yeniden indeksleme her boşluğu onarır.
- Testcontainers Elasticsearch'ü ile @DataElasticsearchTest, arama kodunu gerçek motora karşı test eder.

Sıradaki adım: [Ödevler](#)

6. İleri Okuma

- [Spring Data Elasticsearch Reference · Object Mapping](#)
- [Spring Boot — Elasticsearch](#)
- [Elasticsearch — Language Analyzers](#)
- [Elasticsearch — Bool Query · Highlighting](#)
- [Elasticsearch — Terms Aggregation](#)
- [Spring Framework — Transaction-bound Events](#)